Loading older messages…
Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your State will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page.
…

Here is your website mock-up! It shows the layout of your planned Factor of Safety Calculator, including the navigation between pages and the two dataclasses.

What the mock-up includes
Home page: Introduction and Get Started button.

Calculator page: Input fields for force, area, yield strength, and target factor of safety.

Results page: Calculated stress, factor of safety, and a graph.

History page: Previous calculations.

Calculation Details page: Detailed information about a selected calculation.

State Dataclass: Persistent application state and calculation history.

Calculation Dataclass: Information stored for each calculation.

Before submitting
A couple of things to remember:

Add an about() route to explain the engineering formulas.

Add a clear_history() route for clearing previous calculations.

Label the calculation-details navigation parameter as calculation_index: int so you know which calculation to display.

Suggested caption for your worksheet:

"Website mock-up showing the planned page layouts, navigation flow, user input fields, calculation results, and dataclass structure for my Mechanical Engineering Factor of Safety Calculator."

You can use this as your visual mock-up and make small changes if your instructor wants a hand-drawn version.

ok lets start with the home page code I startedd it with dataclasses from dataclasses import dataclass
from drafter import *
from bakery import assert_equal

@dataclass
class Calculation:
force: float
area: float
yield_strength: float
stress: float
factor_of_safety: float
target_fos: float
assessment: str

@dataclass
class State:
force_input: str
area_input: str
yield_strength: str
target_fos_input: str
calculations: list[Calculation]
…
Great! You've already completed the most important starting step: creating your two dataclasses.

Let's build the home page first, one piece at a time. We'll keep the code beginner-friendly so you understand what each part does.

1. First, make a small correction to your State dataclass

I recommend changing yield_strength to yield_strength_input so the name matches your other input variables.

I also recommend using field(default_factory=list) for your calculation history. This creates an empty list for each new application state.

2. Updated dataclasses
Replace your current code with this:

```
from dataclasses import dataclass, field
from drafter import *
from bakery import assert_equal

@dataclass
class Calculation:
force: float
area: float
yield_strength: float
stress: float
factor_of_safety: float
target_fos: float
assessment: str

@dataclass
class State:
force_input: str = ""
area_input: str = ""
yield_strength_input: str = ""
target_fos_input: str = ""
calculations: list[Calculation] = field(default_factory=list)
```

3. Create your home page
Your home page should:

Welcome the user.

Explain what the calculator does.

Have a button to begin.

Have a short description of the engineering calculations.

Add this below your dataclasses:

```
@route
def home(state: State) -> Page:
return Page(state, [
Heading("Mechanical Engineering Calculator"),

Text("Welcome to the Factor of Safety Calculator!"),

Text("This application helps you calculate normal stress and factor of safety for a mechanical component."),

Text("Enter an applied force, cross-sectional area, and material yield strength to see how safe your component is under the applied load."),

Button("Get Started", calculator),

Text("Learn how engineering forces affect mechanical components!")
])
```

4. Add a temporary calculator page
Since your home page button needs somewhere to go, add this temporary route underneath it:

```
@route
def calculator(state: State) -> Page:
return Page(state, [
Heading("Factor of Safety Calculator"),
Text("Your calculator input fields will go here."),
Button("Back to Home", home)
])
```

5. Start your application
At the bottom of your program, add:

```
start_server(State())
```

Your first milestone You should now have a working home page with a button that navigates to your calculator page.

6. What your home page will look like

Mechanical Engineering Calculator
Factor of Safety Calculator

Welcome!
Calculate normal stress and factor of safety for mechanical components.

What can you do?

Calculate normal stress.

Determine factor of safety.

Compare results with your target.

Visualize how force affects safety.

Get Started

Educational mechanical engineering tool

One thing at a time: Get this home page running before we move on to the calculator inputs.

Once it works, we can build the calculator page together and I'll explain how each Drafter component works.

i kinda like what i got I took your example and did this: @route
def index(state: State) -> Page:
return Page(state, [
"Normal Stress and Factor of Safety Calculator",
"\n",
"Welcome! This calculator helps you calculate normal stress and determine the factor of safety for a material.",
"\n",
"Use this website to enter your information and see your results.",
Button("Open Calculator", "calculator"),
Button("View History", "history"),
Button("About", "about")
])
Honestly, I like your version! It's simple, clean, and does exactly what a home page should do. I would keep most of what you wrote.

There are just two things I'd point out:

Your button navigation: Using "calculator", "history", and "about" is fine if those routes are registered with those names in your Drafter setup.

Your spacing: You can keep the "\n" strings, although you could also use separate text components for more control over spacing.

Your code with a few small improvements

```
@route
def index(state: State) -> Page:
return Page(state, [
Heading("Normal Stress and Factor of Safety Calculator"),

Text("Welcome! This calculator helps you calculate normal stress and determine the factor of safety for a material."),

Text("Use this website to enter your information and see your results."),

Button("Open Calculator", "calculator"),
Button("View History", "history"),
Button("About", "about")
])
```

What I like about your design
The title clearly explains what the website does.

The welcome message explains the purpose.

The three buttons make navigation straightforward.

You aren't adding unnecessary features that will make the project harder.

One question before we continue
Do you want to keep your home page exactly this simple, or would you like to add a small section explaining what normal stress and factor of safety mean?

Either approach is perfectly reasonable for your CS1 project.

```
@route
def calculator(state: State) -> Page:
return Page(state, [
"Normal Stress Calculator",
"\n",
```

"Enter the following information:",
"\n",
"Applied Force (N):",
TextBox("force_input"),
"\n",
"Cross-Sectional Area (mm^2):",
TextBox("area_input"),
"\n",
"Yield Strength (MPa):",
TextBox("yield_strength"),
"\n",
"Target Factor of Safety:",
TextBox("target_fos_input"),
"\n",
Button("Submit", "calculate"),
Button("Back Home", "index"),
])
…
This looks good! You're keeping the calculator straightforward, and you've included all four inputs needed for your engineering calculations.

There is just one important thing to fix before moving forward.

1. Fix your variable name
In your original State dataclass, you have:

yield_strength_input: str

But in your calculator page, you wrote:

TextBox("yield_strength")

These names need to match so Drafter knows where to store the user's input.

I'd change it to:

TextBox("yield_strength_input")

2. Your corrected calculator page

@route
def calculator(state: State) -> Page:
return Page(state, [
"Normal Stress Calculator",

```
"\n",
"Enter the following information:",
"\n",
"Applied Force (N):",
TextBox("force_input"),
"\n",
"Cross-Sectional Area (mm^2):",
TextBox("area_input"),
"\n",
"Yield Strength (MPa):",
TextBox("yield_strength_input"),
"\n",
"Target Factor of Safety:",
TextBox("target_fos_input"),
"\n",
Button("Submit", "calculate"),
Button("Back Home", "index"),
])
```

3. How your inputs connect to State
TextBox name

State attribute

force_input

force_input

area_input

area_input

yield_strength_input

yield_strength_input

target_fos_input

target_fos_input

4. What's next?
Your next route will be calculate.

That function will:

Take the four inputs.

Convert the strings into numbers.

Calculate normal stress.

Calculate factor of safety.

Save the calculation in your history list.

My suggestion: Before we write that function, let's make sure your calculator page works and all four inputs are being stored correctly. Then we'll tackle the engineering math!

This might also interest you.

ok I dont mind this but I think right now can you give me just an example of each page in code

Worked for 12s
Absolutely. Here is an example of each page in your Factor of Safety Calculator, keeping the same simple coding style you've been using.

I'm going to include:

Home page

Calculator page

Calculate function

Results page

History page

Calculation Details page

About page

Clear History function

These are starter examples, not a requirement to copy everything at once.

1. Home page — index()

```
@route
def index(state: State) -> Page:
return Page(state, [
"Normal Stress and Factor of Safety Calculator",
"\n",
"Welcome! This calculator helps you calculate normal stress and determine the factor of safety for a material.",
"\n",
"Use this website to enter your information and see your results.",
Button("Open Calculator", "calculator"),
Button("View History", "history"),
Button("About", "about")
])
```

2. Calculator page — calculator()

```
@route
def calculator(state: State) -> Page:
return Page(state, [
"Normal Stress Calculator",
"\n",
"Enter the following information:",
"\n",
"Applied Force (N):",
TextBox("force_input"),
"\n",
"Cross-Sectional Area (mm^2):",
TextBox("area_input"),
"\n",
"Yield Strength (MPa):",
TextBox("yield_strength_input"),
"\n",
"Target Factor of Safety:",
TextBox("target_fos_input"),
"\n",
Button("Submit", "calculate"),
Button("Back Home", "index")
])
```

3. Calculate function — calculate()
This function processes the information entered by the user.

```
@route
def calculate(state: State) -> Page:
force = float(state.force_input)
area = float(state.area_input)
yield_strength = float(state.yield_strength_input)
target = float(state.target_fos_input)

stress = force / area
fos = yield_strength / stress

if fos >= target:
assessment = "Meets Target"
else:
assessment = "Below Target"

new_calculation = Calculation(
force,
area,
yield_strength,
stress,
fos,
target,
assessment
)

state.calculations.append(new_calculation)

return results(state)
```

This is a basic example. Later, you should add input validation to prevent invalid entries and division by zero.

4. Results page — results()

```
@route
def results(state: State) -> Page:
latest = state.calculations[-1]

return Page(state, [
```

```
"Calculation Results",
"\n",
"Normal Stress:",
str(round(latest.stress, 2)) + " MPa",
"\n",
"Factor of Safety:",
str(round(latest.factor_of_safety, 2)),
"\n",
"Assessment:",
latest.assessment,
"\n",
Button("View History", "history"),
Button("Calculate Again", "calculator"),
Button("Back Home", "index")
])
```

5. History page — history()
This page uses a for loop to display previous calculations.

```
@route
def history(state: State) -> Page:
content = [
"Calculation History",
"\n"
]

if len(state.calculations) == 0:
content.append("You have no previous calculations.")

for calculation in state.calculations:
content.append("Applied Force: " + str(calculation.force) + " N")
content.append("Stress: " + str(round(calculation.stress, 2)) + " MPa")
content.append("Factor of Safety: " + str(round(calculation.factor_of_safety, 2)))
content.append("Assessment: " + calculation.assessment)
content.append("\n")

content.append(Button("Clear History", "clear_history"))
content.append(Button("Back Home", "index"))
content.append(Button("Open Calculator", "calculator"))

return Page(state, content)
```

6. Calculation Details page — calculation_details()
This page displays the most recent calculation in greater detail. This is a simplified starting point; later you could let users select any calculation from history.

```
@route
def calculation_details(state: State) -> Page:
latest = state.calculations[-1]

return Page(state, [
"Calculation Details",
"\n",
"Applied Force: " + str(latest.force) + " N",
"Cross-Sectional Area: " + str(latest.area) + " mm^2",
"Yield Strength: " + str(latest.yield_strength) + " MPa",
"Normal Stress: " + str(round(latest.stress, 2)) + " MPa",
"Factor of Safety: " + str(round(latest.factor_of_safety, 2)),
"Assessment: " + latest.assessment,
"\n",
Button("Back to History", "history")
])
```

7. About page — about()

```
@route
def about(state: State) -> Page:
return Page(state, [
"About This Calculator",
"\n",
"This website was created to help users understand normal stress and factor of safety in mechanical components.",
"\n",
"Normal Stress = Force / Cross-Sectional Area",
"\n",
"Factor of Safety = Yield Strength / Normal Stress",
"\n",
"The calculator is designed for educational purposes.",
Button("Back Home", "index")
])
```

8. Clear History — clear_history()

```
@route
def clear_history(state: State) -> Page:
state.calculations = []

return Page(state, [
"Calculation History Cleared!",
"\n",
"Your previous calculations have been removed.",
Button("Back to History", "history"),
Button("Back Home", "index")
])
```

How all your pages connect
index() — Home

calculator() — User Inputs

calculate() — Engineering Math

results() — Results

history() — Previous Calculations

Additional pages:

calculation_details()

about()

clear_history()

A couple of important notes before using this code:

Your State dataclass must use the same names as the TextBox fields, especially yield_strength_input.

The results() and calculation_details() pages assume at least one calculation exists. Add checks before accessing [-1].

This example assumes your Drafter setup supports the string route names used in your buttons, as in your existing code.

The graph can be added to the results page later using matplotlib.

My recommendation: Don't try to finish all eight functions at once. Build and test each route individually. That will make debugging much easier and help you understand your own project.

```
@route
def calculator(state: State) -> Page:
return Page(state, [
"Normal Stress Calculator",
"\n",
"Enter the following information:",
"\n",
"Applied Force (N):",
TextBox("force_input"),
"\n",
"Cross-Sectional Area (mm^2):",
TextBox("area_input"),
"\n",
"Yield Strength (MPa):",
TextBox("yield_strength"),
"\n",
"Target Factor of Safety:",
TextBox("target_fos_input"),
"\n",
Button("Submit", "calculate"),
Button("Back Home", "index"),
])
@route
def calculate(state: State, force_input: str, area_input: str,
yield_strength: str, target_fos_input: str) -> Page:

try:
force = float(force_input)
area = float(area_input)
yield_strength = float(yield_strength)
target_fos = float(target_fos_input)

except ValueError:
return Page(state, [
"Please enter valid numbers.",
Button("Try Again", "calculator")
])

if force <= 0 or area <= 0 or yield_strength <= 0 or target_fos <= 0:
```

```
return Page(state, [
"All inputs must be greater than zero.",
Button("Try Again", "calculator")
])

stress = calculate_stress(force, area)

factor_of_safety = calculate_factor_of_safety(yield_strength, stress)

assessment = check_target(factor_of_safety, target_fos)

calculation = Calculation(
force,
area,
yield_strength,
stress,
factor_of_safety,
target_fos,
assessment
)

state.calculations.append(calculation)

return Page(state, [
"Your calculation is complete!",
"\n",
Button("View Results", "results"),
Button("Home", "index")
])
@route
def results(state: State) -> Page:
calculation = state.calculations[-1]

return Page(state, [
"Calculation Results",
"\n",
"Applied Force: " + str(calculation.force) + " N",
"\n",
"Cross-Sectional Area: " + str(calculation.area) + " mm^2",
"\n",
"Yield Strength: " + str(calculation.yield_strength) + " MPa",
"\n",
"Normal Stress: " + str(round(calculation.stress, 2)) + " MPa",
"\n",
```

```
"Factor of Safety: " + str(round(calculation.factor_of_safety, 2)),
"\n",
"Target Factor of Safety: " + str(calculation.target_fos),
"\n",
calculation.assessment,
"\n",
Button("Calculate Again", "calculator"),
Button("View History", "history"),
Button("Home", "index")
])
@route
def history(state: State) -> Page:
content = [
"Calculation History",
"\n"
]

if len(state.calculations) == 0:
content.append("No calculations have been made yet.")
else:
for index in range(len(state.calculations)):
calculation = state.calculations[index]

content.append("Applied Force: " + str(calculation.force) + " N")
content.append("Normal Stress: " + str(round(calculation.stress, 2)) + " MPa")
content.append("Factor of Safety: " + str(round(calculation.factor_of_safety, 2)))
content.append(calculation.assessment)

content.append(Button("View Details", "calculation_details", [Argument("calculation_index",
index)]))
content.append("--------------------")
content.append(Button("Back Home", "index"))
content.append(Button("Calculate Again", "calculator"))
content.append(Button("Clear History", "clear_history"))

return Page(state, content)
@route
def calculation_details(state: State, calculation_index: int) -> Page:
calculation = state.calculations[calculation_index]

return Page(state, [
"Calculation Details",
"\n",
"Applied Force: " + str(calculation.force) + " N",
```

```
"\n",
"Cross-Sectional Area: " + str(calculation.area) + " mm^2",
"\n",
"Yield Strength: " + str(calculation.yield_strength) + " MPa",
"\n",
"Normal Stress: " + str(round(calculation.stress, 2)) + " MPa",
"\n",
"Factor of Safety: " + str(round(calculation.factor_of_safety, 2)),
"\n",
"Target Factor of Safety: " + str(calculation.target_fos),
"\n",
calculation.assessment,
"\n",
Button("Back to History", "history"),
Button("Home", "index")
])
@route
def clear_history(state: State) -> Page:
state.calculations = []

return Page(state, [
"Your calculation history has been cleared!",
"\n",
Button("Back to History", "history"),
Button("Home", "index")
])
@route
def about(state: State) -> Page:
return Page(state, [
"About This Calculator",
"\n",
"This website is designed to calculate normal stress and factor of safety for engineering materials.",
"\n",
"Users can enter an applied force, cross-sectional area, yield strength, and target factor of safety.",
"\n",
"The calculator uses these values to determine the normal stress and factor of safety.",
"\n",
"Users can also save, view, and clear their previous calculations.",
"\n",
Button("Back Home", "index"),
Button("Open Calculator", "calculator")
])
```

start_server(State("", "", "", "", []))
…

Worked for 11s
This is coming together really well! You're now building an actual application rather than just a collection of pages.

I especially like that you've included:

Input validation using try/except.

Separate functions for the engineering calculations.

A calculation history using your Calculation dataclass.

A loop to display previous calculations.

A details page where users can inspect individual calculations.

A button to clear the history.

Your overall structure is good, and I would keep it. I would make a few small corrections before moving forward.

1. Important things to fix
A. Make sure your input names are consistent
You currently have:

TextBox("yield_strength")

And your calculate route accepts:

yield_strength: str

Those two match, so that part is correct.

However, your original State dataclass used yield_strength_input. You have two choices:

Keep your current yield_strength naming in the TextBox and calculate route.

Change both to yield_strength_input.

I recommend the second option to keep your naming consistent with your other inputs.

B. You need three calculation functions
Your code calls these functions:

stress = calculate_stress(force, area)

factor_of_safety = calculate_factor_of_safety(yield_strength, stress)

assessment = check_target(factor_of_safety, target_fos)

But you haven't defined them yet.

Add these above your routes:

```
def calculate_stress(force: float, area: float) -> float:
return force / area


def calculate_factor_of_safety(yield_strength: float, stress: float) -> float:
return yield_strength / stress


def check_target(factor_of_safety: float, target_fos: float) -> str:
if factor_of_safety >= target_fos:
return "Your component meets the target factor of safety."
else:
return "Your component is below the target factor of safety."
```

These functions keep your engineering math separate from your website pages, which makes your program easier to understand and troubleshoot.

2. Add one improvement to your calculate route
After converting your inputs to numbers, I recommend saving them into your State:

```
state.force_input = force_input
state.area_input = area_input
state.yield_strength_input = yield_strength
state.target_fos_input = target_fos_input
```

This way, your State attributes actually get updated when someone uses the calculator.

3. Add protection to your results page
Currently, you have:

calculation = state.calculations[-1]

This works as long as there is at least one calculation. However, if someone visits the results page before calculating anything, the program will produce an error.

Add an if statement:

```
if len(state.calculations) == 0:
return Page(state, [
"No calculations available.",
Button("Open Calculator", "calculator"),
Button("Home", "index")
])
```

Put this before calculation = state.calculations[-1].

Do something similar in calculation_details() to check that the requested index is valid.

4. My assessment of your project so far

Requirement	Status
Two dataclasses	Done
7 routes minimum	Done
5 pages minimum	Done
3 user inputs minimum	Done
3 if statements minimum	Done

Loop through calculations

Done

Engineering calculations

Needs helper functions

Input validation

Done

5. What I would work on next
I would avoid adding any more pages right now. You already have enough routes.

Instead, I would work in this order:

Get the three engineering calculation functions working.

Test the calculator with sample numbers.

Make sure calculations appear in your history.

Test the clear history button.

Add your matplotlib graph.

Let's make sure you understand the engineering math before adding more features.

For example, if a user enters:

Force = 5,000 N

Area = 100 mm²

Yield strength = 250 MPa

Target factor of safety = 2.0

Your program should calculate:

Normal stress = 50 MPa

Factor of safety = 5.0

Assessment = Meets the target factor of safety.

That is a good test case to use when debugging your program.

This might also interest you.

Context window full
Start a new chat with an automatic summary of this conversation, or upgrade to continue here with more context available in every chat.

0